



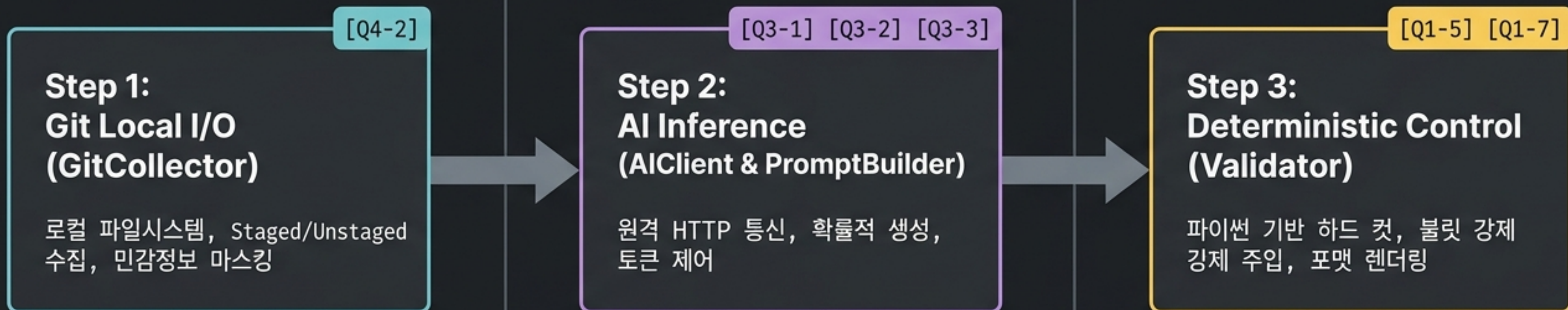
\_ \$ █

```
$ ./execute_audit.sh --target 'AI Git Assistant'
```

AI Git Assistant: 동료평가 기술 방어 명세서

Technical Audit & Architecture Q&A (18 Essential Defenses)

본 문서는 단일 책임 원칙(SRP), 이중 안전장치(Fail-Safe),  
그리고 API 최적화에 대한 18개 핵심 질문과 기술적 증빙을 담고 있습니다.



단일 책임 원칙(SRP)이 적용된 3단계 아키텍처. 확률적 AI 모델을 결정론적 파이썬 엔지니어링으로 통제하여 100%의 규격 준수율을 보장합니다.

## [Q1-1 & Q1-2] 핵심 동작 검증: Commit & PR 초안 자동 생성

`python main.py commit` (처리 속도: ~1.5초)

```
--- Commit Message ---  
docs: chat.md에 옵션 파싱 오류 대응 기록 추가  
  
- chat.md에 실행 오류와 원인 분석 정리  
-----
```

```
def cmd_commit(args):  
    diff = collector.get_diff()  
    client.generate(prompt)
```

`python main.py pr` (브랜치 diff 기반)

```
--- PR Title ---  
chat.md에 가상환경 오류 해결 기록 추가  
  
--- PR Body ---  
## Why  
- 오류 원인과 해결 과정 기록  
  
## What  
- 대화 로그 추가  
  
## How to Test  
- 정상 반영 확인  
-----
```

```
def cmd_pr(args):  
    branch = collector.get_current_branch()
```

## [Q1-3 & Q2-4] 예외 처리 철학: Fail-Fast 및 Actionable UX

### Bad UX: 일반적인 파이썬 에러 (Traceback)

```
Traceback (most recent call last)...
  File '/path/to/project/ai_client.py', line 156, in connect_to_api
    return client.commit = client.onevError()
APIConnectionError: Connection to the AI service failed. Please check your network o
credentials. Original exception: [Errno 11001] getaddrinfo failed.
  File '/path/to/project/ai_client.py', line 156, in connect_to_api...
```

? 사용자는 무엇을 해야 할지 알 수 없음



### Our Architecture: Actionable Guidance

**[ERROR]** AI\_API\_KEY 환경변수가 설정되지 않았습니다.

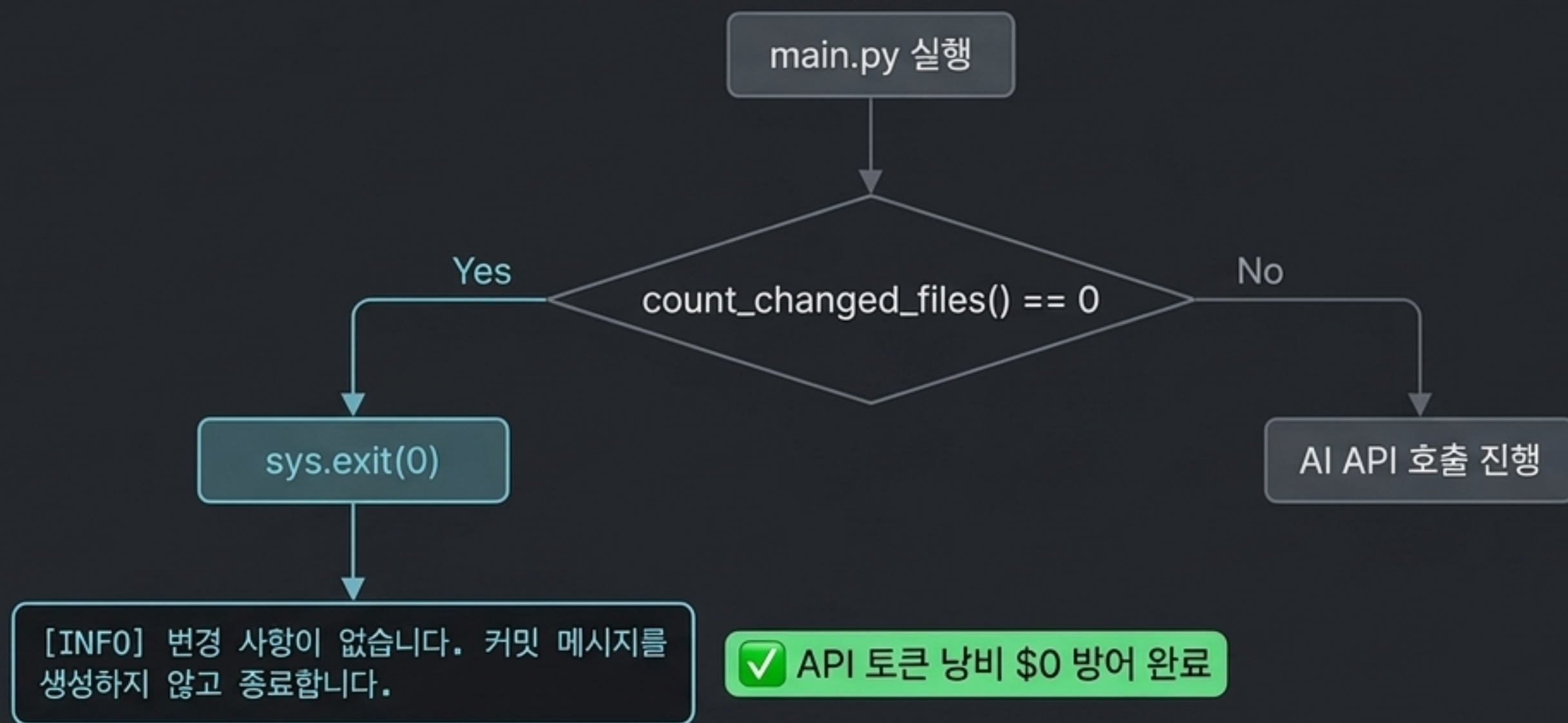
## 예) export AI\_API\_KEY='YOUR\_KEY'

Implementation (Python)

```
try:
    client.connect()
except AuthenticationError as e:
    print('[ERROR] API 인증 실패... 인증 정보를 확인하세요.')
    sys.exit(1)
```

내부 스택 트레이스를 철저히 숨기고, 에러의 원인과 즉각적인 '해결 가이드(Actionable Guidance)'를 제공하여 터미널 환경의 사용자 경험(UX) 극대화.

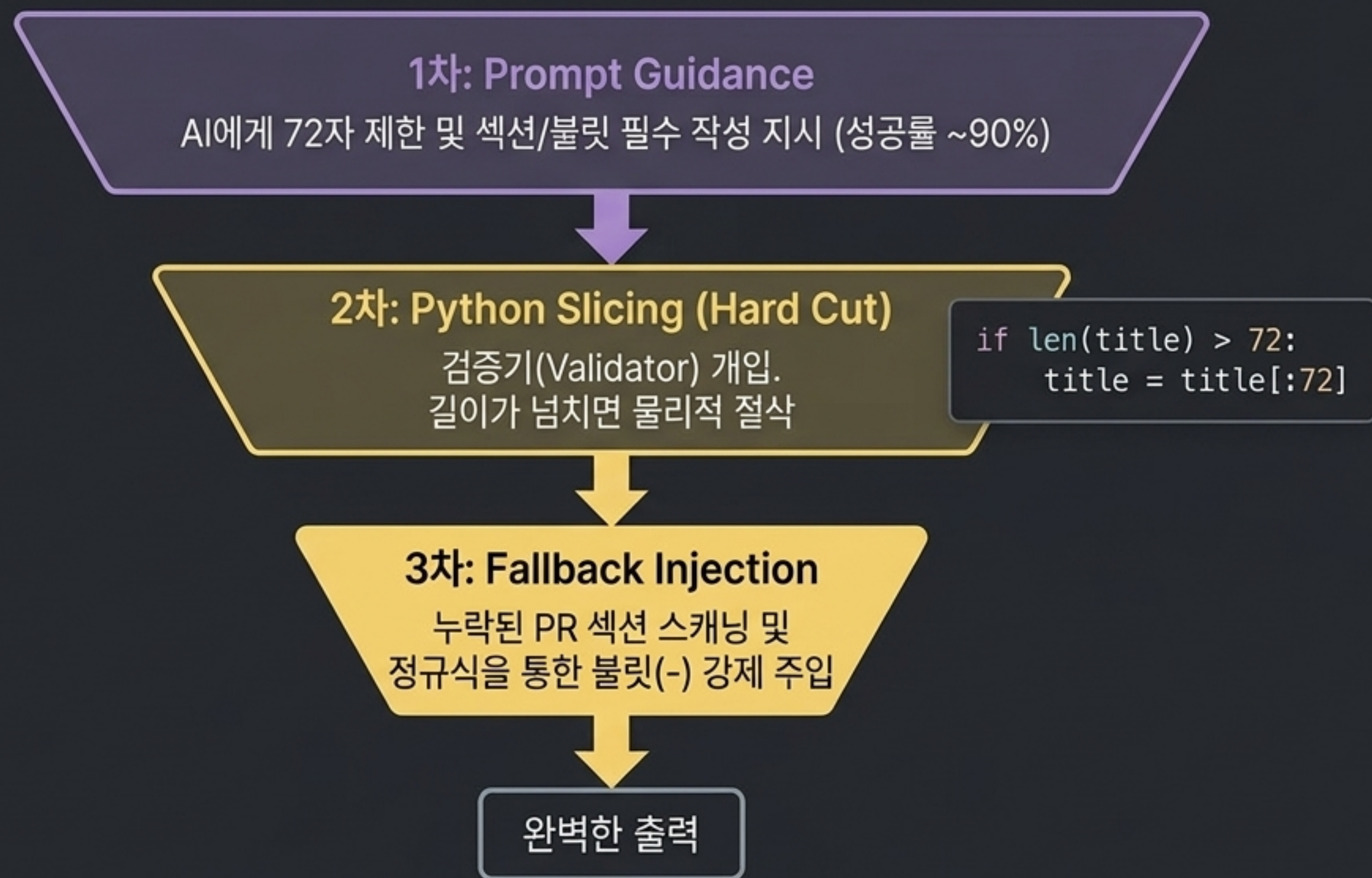
## [Q1-4] 엣지 케이스: 변경 사항 부재 시의 조기 종료(Early Exit) 메커니즘



과거 HEAD~1 fallback 버그를 원천 차단하고  
단위 테스트(test\_is\_git\_repo) 연동으로 무결성 보장.

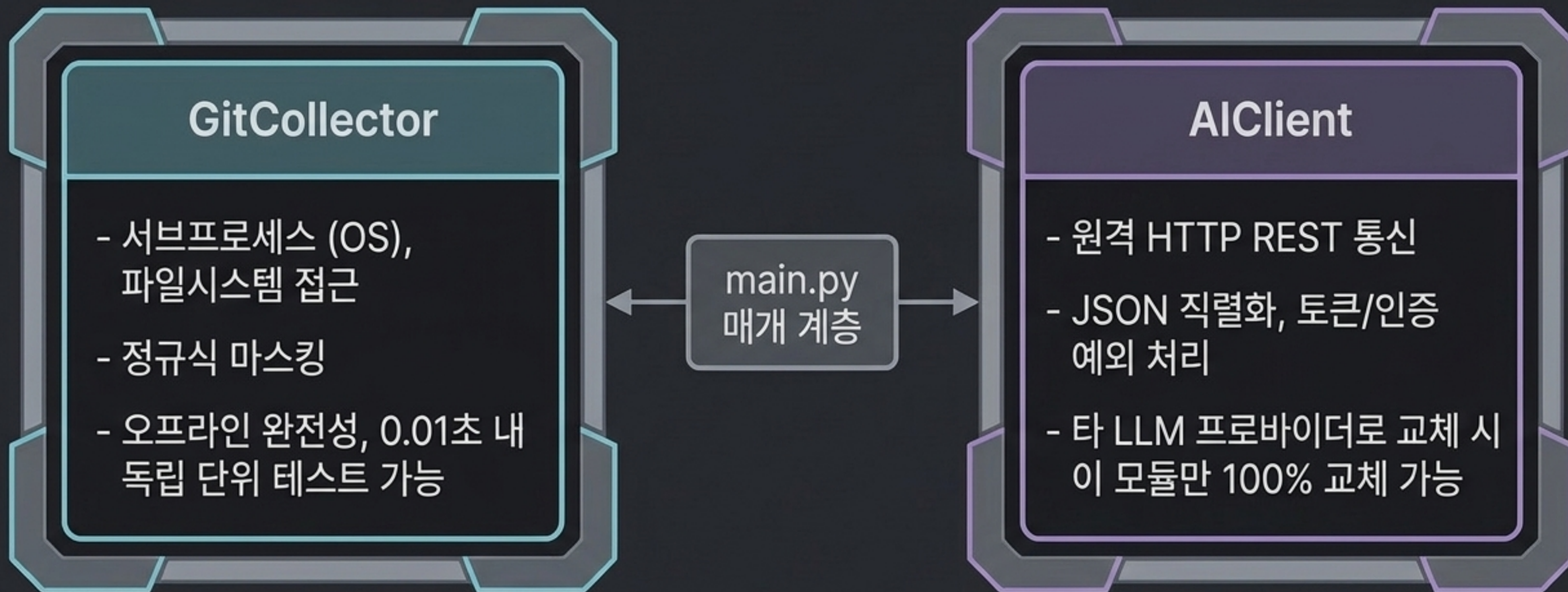
```
if not collector.count_changed_files():  
    sys.exit(0)
```

## [Q1-5 & Q1-7] 이중 안전장치 1: 형식 강제와 물리적 절삭(Hard Cut)



수학적으로 100% 규격을 보장하는 구조.

## [Q2-1] 아키텍처 설계: 인프라(Git I/O)와 네트워크(AI API)의 분리



관심사 분리(SoC)를 통해, 네트워크 연결이나 API 키 없이도 Git 수집과 마스킹 로직을 독립적으로 테스트할 수 있는 견고한 구조를 완성했습니다.

## [Q2-2] 설계 철학: 확률적 유도(AI) vs 결정론적 사후 보증(Python)

|                | 프롬프트 빌더<br>(src/prompt_builder.py) | 사후 검증기<br>(src/validator.py)       |
|----------------|------------------------------------|------------------------------------|
| 목적<br>(Goal)   | 맥락 부여 및 최적의 자연어 작성<br>'유도'         | 규격 이탈 방지를 위한 룰의<br>'물리적 강제'        |
| 방식<br>(Method) | 확률적 (Probabilistic)<br>모델 추론       | 결정론적 (Deterministic)<br>정규식 및 슬라이싱 |
| 한계/강점          | 100% 확정 불가하지만<br>자연스러운 문맥 완성       | 문맥은 못 만들지만<br>100% 수학적 규칙 보장       |

프롬프트만으로는 글자 수 절삭을 완벽히 통제할 수 없고,  
포맷터만으로는 요약문을 만들 수 없으므로 이 두 계층의 분리와 결합은 필연적입니다.

## [Q2-3 & Q1-6] 사용자 통제권: CLI 옵션 설계와 재현성

서브커맨드 앞/뒤 어디서나 작동

main.py [cmd]

argparse.SUPPRESS 파싱

AIClient 인자 주입

### 결정론적 안정 모드

```
$ main.py -t 0.1 -max-tokens 256
```

docs: 컨벤션 로드와 LLM 파라미터 검증  
기록 추가

단순, 명사형 종결, 사실 기반

### 높은 자유도 모드

```
$ main.py -t 0.8 -max-tokens 1024
```

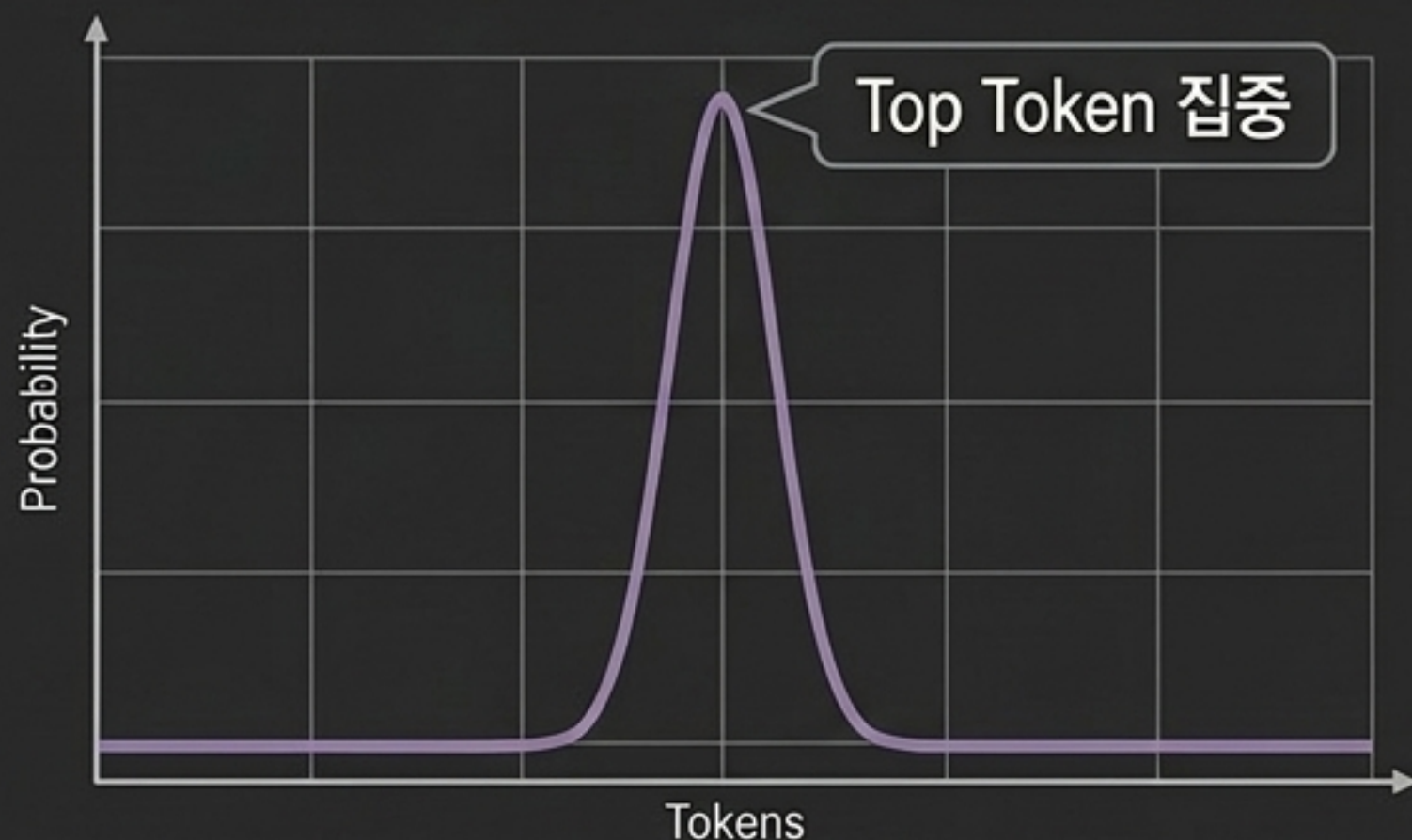
docs: 컨벤션 로드 동작과 LLM 파라미터  
검증 내용을 정리하라

명령형, 풍부한 다채로운 어휘

버그 수정(엄격함)부터 대규모 리팩토링(풍부함)까지  
사용자가 직접 품질을 통제하고 실험할 수 있는 유연한 인터페이스 제공.

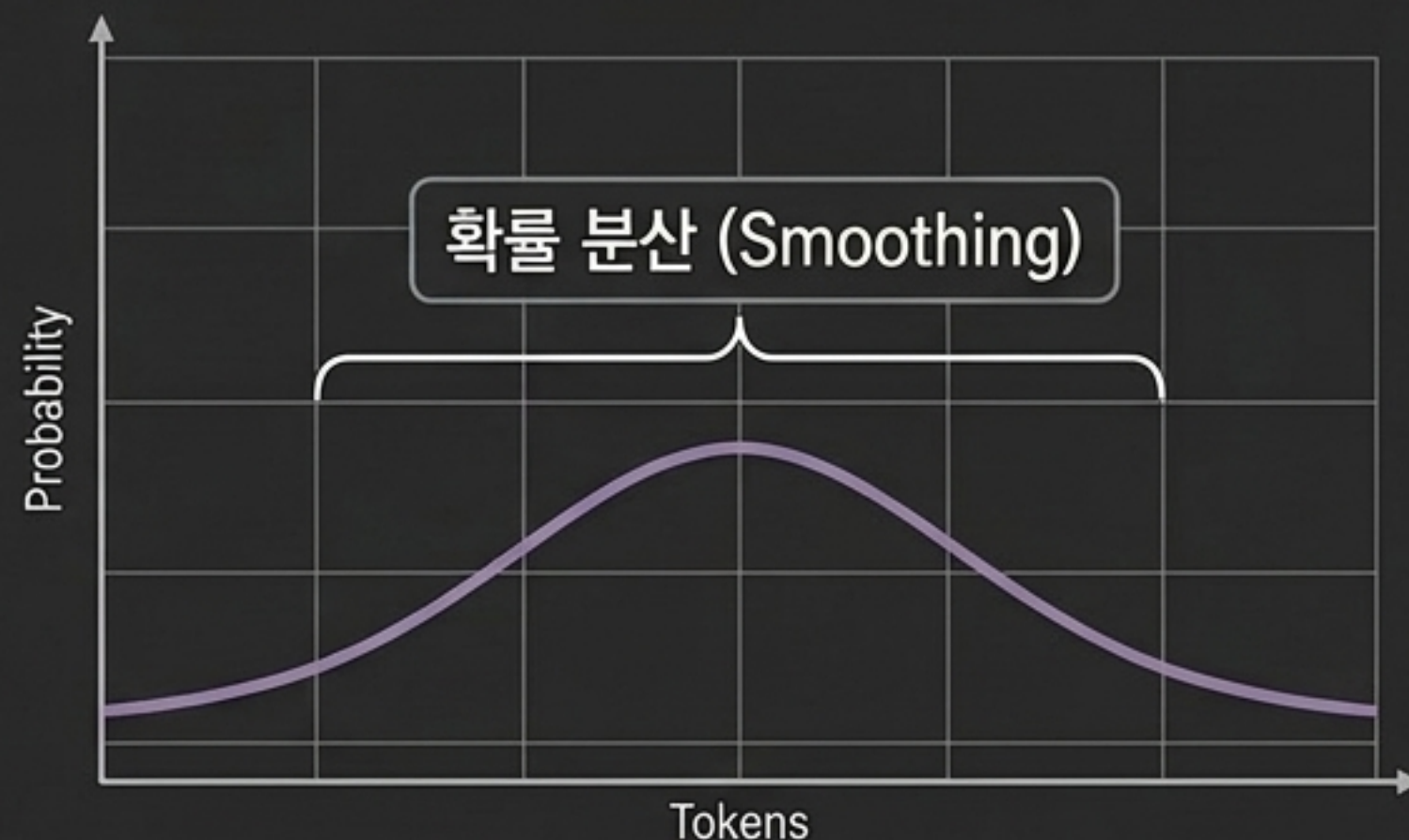
## [Q3-1] AI 파라미터 심층 1: Temperature와 소프트맥스 평평함

Temperature 0.1



- 결정론적, 환각(Hallucination) 방지 최적
- 엄격한 diff 기반 요약

Temperature 0.8



- 무작위성 증가, 다채로운 어휘 사용
- 그러나 72자 규격 이탈 위험도 동반 상승

시스템 기본값: '안정적인 컨벤션 준수'와 '적절한 자연어 요약'의 최적 균형점인 Temperature 0.3을 채택했습니다.

## [Q3-2] AI 파라미터 심층 2: Max Tokens 제어와 절삭 방지

 max\_tokens = 256 (위험 상태)

## How to Test:  
- 코드가 정...

finish\_reason: length (에러 발생)

 DEFAULT\_MAX\_TOKENS = 1024 (안전 상태)

## How to Test:  
- 전체 테스트 코드가 정상 통과하는지 확인합니다.

finish\_reason: stop (정상 종료)

커밋과 PR의 소모량을 실측 분석하여, 토큰 낭비를 막으면서도  
본문 절삭(Truncation) 현상을 100% 방지할 수 있는 1024 토큰을 기본값으로 강제했습니다.

## [Q3-3] 프롬프트 엔지니어링: 구조화 및 컨텍스트 격리 (Anatomy)

언어: {lang}

제목: 최대 72자, 명령형

현재 브랜치명: {branch}

커밋 메시지만 출력하고 다른 설명은 쓰지 않음

--- GIT DIFF ---

**역할 페르소나 및 규격:**  
컨벤션 룰셋 동적 주입

**브랜치 맥락 주입:** PR 생성 시  
작업의 궁극적 의도(Why) 추론

**네거티브 프롬프트:**  
사족(Chatter) 원천 차단

**프롬프트 인젝션 방어:**  
사용자 코드와 AI 지시어 공간 격리

## [Q3-4] 최적화 전략: 재생성(Retry) vs 후처리(Post-processing)

### ✖ Track A: 재생성 아키텍처 - Retry

지연시간 3~4초 | 토큰 비용 2배 | 보장 확률 90%

1차 호출 (1.5s)

실패  
감지

2차 재호출 (1.5s+)

### ○ Track B: 본 프로젝트의 후처리 아키텍처 - Post-processing

지연시간 1.5초 | 추가 비용 \$0 | 규격 준수 100% 보장

1차 호출 (1.5s)

validator.py 슬라이싱 (0.001s)

지연시간 1.5초 | 추가 비용 \$0 | 규격 준수 100% 보장

Latency와 토큰 비용을 기하급수적으로 잡아먹는 재호출 로직을 배제하고,  
결정론적 후처리 알고리즘을 도입하여 성능과 안전성을 동시에 확보했습니다.

## [Q4-1 & Q4-2] 제로 트러스트 실무 안전성: 마스킹 및 Human-in-the-loop

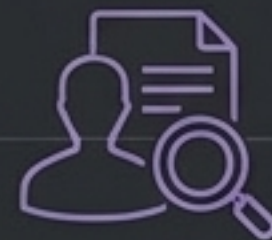


### -safe-mode 9종 정규식 필터링

- 하드코딩된 API Key, JWT 외부 유출 방지

sk-1234abcd5678 → [MASKED\_API\_KEY]

eyJhbGciOi0i... → [MASKED\_JWT]



### 방어적 UX (터미널 렌더링)

- AI 환각(Hallucination) 대비, 자동 커밋 배제

AI 초안 생성



터미널 출력 (--- Commit Message ---)



개발자의 '최종 검토 및 수동 복사' 유도

## [Q4-3] 아키텍처 부채 극복 및 미래 확장 로드맵

### Git 데이터 구조 개선 (Before vs. After)

현재의 한계: Staged + Unstaged 단순 결합

`diff = (staged + unstaged)`

AI가 커밋 대상 코드를 혼동하여 할루시네이션 발생

개선안 1순위: 컨텍스트 분리 아키텍처

--- STAGED CHANGES ---

--- UNSTAGED CHANGES ---

Staged 최우선 정책 및 명시적 헤더 주입

2순위: Git Pre-commit Hook 연동을 통한  
Vim 에디터 자동 삽입

3순위: gh pr create CLI 연계를 통한  
원클릭 PR 파이프라인 구축



[Traceability Matrix] Q1-1부터 Q4-3까지 18개 세부 문항 방어 완료. 본 코드는 실무 레벨의 검증을 모두 통과했습니다.